

REPORTE:

Implementación de patrones estructurales en una aplicación móvil desarrollada en Flutter

Luisa Fernanda Hernández Hdez.

8 de mayo de 2026



Universidad Veracruzana
Ingeniería de Software

Índice

1. Introducción	3
2. Descripción del Proyecto: Aplicación en Flutter y API REST	3
3. Implementación de Patrones Estructurales en la aplicación de Flutter	3
3.1. Patrón Adapter	3
3.2. Patrón Bridge	4
3.3. Patrón Composite	4
3.4. Patrón Decorator	5
3.5. Patrón Facade	5
3.6. Patrón Flyweight	5
3.7. Patrón Proxy	6
4. Conclusión: Importancia y Beneficios	6

1. Introducción

Los patrones de diseño estructurales se enfocan en la composición de clases y objetos para formar estructuras de software más complejas y eficientes. Su objetivo primordial es facilitar que diferentes partes de un sistema, a menudo con interfaces incompatibles, puedan trabajar juntas de manera armónica sin comprometer la integridad del sistema.

A diferencia de los patrones creacionales, que gestionan el proceso de instanciación, los estructurales utilizan la herencia y la composición para simplificar el diseño y reducir el acoplamiento. Permiten que los desarrolladores identifiquen una forma sencilla de realizar relaciones entre entidades, lo que resulta en sistemas más fáciles de mantener y escalar.

2. Descripción del Proyecto: Aplicación en Flutter y API REST

El sistema desarrollado representa una solución integral para la gestión de interacciones sociales (tweets, likes y comentarios). La arquitectura se divide en dos componentes principales:

1. **Backend (API REST):** Construido sobre el ecosistema de Java con Spring Boot. Utiliza una base de datos PostgreSQL y seguridad basada en JWT. El despliegue se realizó mediante contenedores Docker, garantizando un entorno de ejecución aislado y escalable en la nube (Render).
2. **Frontend (Flutter):** Una aplicación móvil y web desarrollada bajo el paradigma de programación reactiva. Esta aplicación consume los recursos de la API de forma asíncrona, permitiendo la visualización de datos dinámicos y la persistencia de interacciones del usuario en tiempo real.

3. Implementación de Patrones Estructurales en la aplicación de Flutter

En Flutter, donde la composición es el principio rector (todo es un *Widget*), los patrones estructurales permiten organizar la UI y la lógica de negocio de manera profesional.

3.1. Patrón Adapter

Se utiliza para transformar los datos crudos (JSON) de la API de Spring Boot en objetos de modelo de Dart.

```
class TweetAdapter {  
    static Tweet fromJson(Map<String, dynamic> json) {  
        return Tweet(  
            id: json['id'],
```

```

        content: json['text_content'], // Adapta 'text_content'
        a 'content'
        author: json['user']['name'],
    );
}
}

```

Listing 1: Ejemplo de Adapter para Modelos

3.2. Patrón Bridge

Separa la abstracción del servicio de su implementación específica por plataforma.

```

abstract class NotificationImpl {
    void send(String msg);
}

class AndroidNotification extends NotificationImpl {
    @override
    void send(String msg) => print("Push Android: $msg");
}

class WebNotification extends NotificationImpl {
    @override
    void send(String msg) => print("Toast Web: $msg");
}

```

Listing 2: Ejemplo de Bridge para Notificaciones

3.3. Patrón Composite

Es la base de Flutter; permite tratar widgets simples y complejos de forma uniforme.

```

Widget build(BuildContext context) {
    return Column( // Widget compuesto
        children: [
            Text("Tweet de Luisa"), // Widget simple
            Icon(Icons.favorite),    // Widget simple
        ],
    );
}

```

Listing 3: Uso de Composite en UI

3.4. Patrón Decorator

Añade responsabilidades adicionales a un widget envolviéndolo en otro (Wrapper).

```

Widget decoratedImage(Widget image) {
    return Container(
        decoration: BoxDecoration(

```

```

        border: Border.all(color: Colors.blue, width: 2),
        borderRadius: BorderRadius.circular(10),
      ),
      child: image, // El widget original es "decorado"
    );
  }

```

Listing 4: Ejemplo de Decorator con Container

3.5. Patrón Facade

Proporciona una interfaz simplificada para el subsistema de red (API de Render).

```

class InteractionFacade {
  final ApiService _api = ApiService();

  Future<void> likeTweet(int id) async {
    await _api.post('/interactions/react/$id', body: 1);
  }
}

```

Listing 5: Facade para Servicios API

3.6. Patrón Flyweight

Optimiza la memoria compartiendo instancias de objetos pesados o repetitivos.

```

class TextStyleFactory {
  static const TextStyle tweetStyle = TextStyle(
    fontSize: 14,
    fontWeight: FontWeight.bold,
    color: Colors.black,
  );
  // Se reutiliza la misma instancia en todos los tweets
}

```

Listing 6: Uso de Flyweight en Estilos

3.7. Patrón Proxy

Actúa como un sustituto de un objeto para controlar el acceso o mostrar un estado de carga.

```

Image.network(
  'https://render-api.com/image.png',
  loadingBuilder: (context, child, progress) {
    if (progress == null) return child;
    return CircularProgressIndicator(); // Proxy visual
    mientras carga
  },
);

```

4. Conclusión: Importancia y Beneficios

La adopción de patrones estructurales trasciende la simple organización del código; es una garantía de calidad técnica. Sus beneficios principales incluyen:

- **Flexibilidad:** Permiten realizar cambios profundos en una parte del sistema sin afectar el resto.
- **Reutilización:** Facilitan el uso de componentes existentes en nuevas estructuras.
- **Legibilidad:** Establecen un estándar de comunicación claro para el equipo de desarrollo.

En conclusión, la correcta estructuración del software asegura que el proyecto sea resiliente, capaz de evolucionar ante nuevos requerimientos y técnicamente sostenible a largo plazo.